

1) Sistemas Operativos - Generalidades

- ¿Quién es el encargado de administrar el tiempo de la CPU y cuál es su función principal dentro del sistema?
- Explique a que se denomina “Contexto de Ejecución”. ¿Cuáles son las variables intervinientes?
- ¿A qué se denomina Sistema Operativo de Tiempo Real? ¿Por qué usar un Sistema Operativo de Tiempo Real? (Pag.95)

2) Kernel de FreeRTOS

- Explique y desarrolle las características de una “Tarea” en FreeRTOS. Proponga un ejemplo de una tarea manteniendo la sintaxis correcta.
- Explique y desarrolle los “Estados de una Tarea” y sus transiciones. Indique las funciones que permiten las transiciones.
- ¿A qué nos referimos con el concepto de “tareas de procesamiento continuo”? ¿y a que nos referimos con el concepto de “tareas manejadas por eventos”?
- ¿Cuáles son los tipos de eventos por los que una tarea puede estar esperando al entrar en el estado bloqueado? Explicar.
- ¿Cuál es la función de la IDLE TASK? ¿Puede el procesador no estar ejecutando ninguna tarea en algún momento? Explicar.

3) Desarrolle como es el almacenamiento de datos en el recurso “Cola” proporcionado por el Kernel.

4) Explique utilizando un ejemplo con código el porqué es común que el recurso Cola posea múltiples tareas escritoras y una sola tarea lectora. Explique el proceso de bloqueo por escritura y bloqueo por lectura en el recurso Cola.

5) A la hora de enviar datos compuestos: explique cómo se lleva a cabo dicho proceso y por qué no genera conflictos con la definición de la macro del recurso. ¿Cuál sería la implementación si los datos fueran “grandes”, lo que ocasionaría un problema con la memoria RAM? (Pag. 91)

6) Explicar el funcionamiento del esquema de la fig 1:

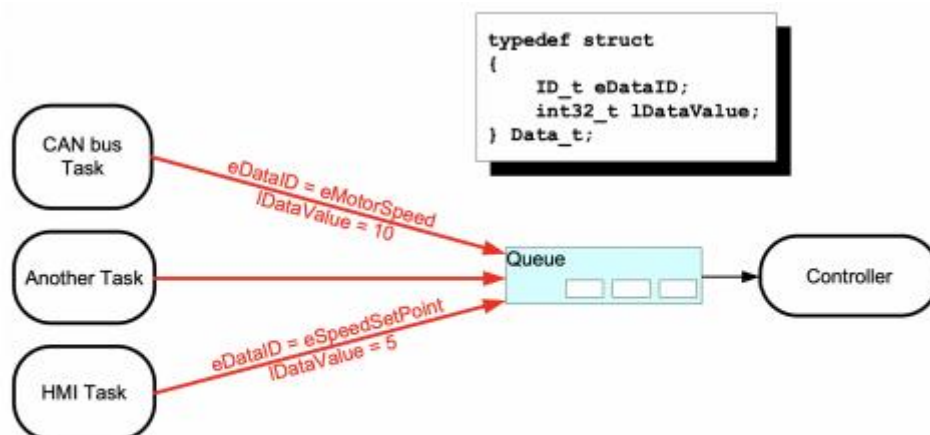


Fig. 1. Recibir datos de múltiples fuentes

7) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Semáforos Mutex”.

- 8) Desarrollar un programa ejemplo que permita comprender el concepto y uso de “Inversión de Prioridad”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.
- 9) Explicar el concepto de “Herencia de Prioridad” utilizando diagramas gráficos que permitan comprenderlo.
- 10) Desarrollar un programa ejemplo que permita comprender el concepto de “Punto Muerto”. Como mínimo se deben utilizar dos tareas y una cola gestionando un recurso del microcontrolador.

Respuestas:

- 1)
- a) El encargado de administrar el tiempo de la CPU es el Scheduler que decide en cada momento qué tarea se va a ejecutar, y por cuánto tiempo, con el objetivo de maximizar el uso del CPU para obtener un funcionamiento eficiente del sistema.
 - b) Para realizar un cambio de una a tarea a otra, es necesario guardar el estado actual de la tarea que se estaba ejecutando y cargar el estado de la tarea que se va a ejecutar. El contexto de ejecución contiene toda la información necesaria para realizar este cambio sin que se produzcan errores. Las variables que intervienen son:
 - Los registros internos del microprocesador.
 - El contador de programa.
 - La pila, que se usa para la llamada a funciones y para almacenar las variables locales.
 - c) Un Sistema Operativo de Tiempo Real es un sistema basado en la ejecución de tareas, las cuales pueden ser planificadas mediante el uso de prioridades y ejecutadas en intervalos de tiempo específicos. Cuenta con la característica de que elimina partes del sistema operativo que no se usen para ahorrar memoria. Además, no se protegen frente a errores de las aplicaciones, esto es para simplificar el diseño y el hardware necesario, lo que lo hace muy bueno para sistemas embebidos basados en microcontrolador.
- 2)
- a) Las tareas se implementan como funciones de C. Solo se distinguen por su prototipo, que debe retornar vacío (void) y tomar un parámetro de puntero vacío (void *).

Ejemplo de prototipo: void ATaskFunction(void *pvParameters);

Cada tarea es un pequeño programa en sí mismo. Tiene un punto de entrada, se ejecuta normalmente de forma continua en un bucle infinito y no se cierra. No debe permitirse que las tareas de FreeRTOS finalicen su ejecución. No deben contener una instrucción "return" ni debe permitirse que se ejecuten al finalizar la función. Si una tarea ya no es necesaria, debe eliminarse explícitamente por medio de la API provista.

Se pueden crear múltiples tareas a partir de definición de función de tarea. Cada tarea creada es una instancia de ejecución independiente, con su propia pila y su propia copia de todas las variables (pila) automáticas definidas en la tarea misma.

Ejemplo de tarea:

```
void ATaskFunction( void *pvParameters )
{

/*Se pueden declarar variables en las tareas como en las funciones*/
int32_t lVariableExample = 0;

/*Se implementa un loop para que no se salga de la tarea*/

While(1)
{
/*Aquí va el código que ejecutará la tarea*/
}

/*Si la tarea sale del loop entonces se debe eliminar la misma antes de que finalice*/

vTaskDelete( NULL );

}
```

b) Cada tarea puede estar en uno de los siguientes tres estados:

Ejecución (Running): El microprocesador la está ejecutando. Sólo puede haber una tarea en este estado.

Bloqueado (Blocked): Se dice que una tarea que está a la espera de un evento se encuentra en estado Bloqueado, que es un subestado de “Not running”. Los elementos que pueden bloquear una tarea en FreeRTOS son colas, semáforos binarios, exclusiones mutuas, exclusiones mutuas recursivas, grupos de eventos y notificaciones directas en la tarea para crear eventos de sincronización.

Suspendido (Suspended): El estado suspendido es también un subestado de “Not running”. Las tareas que se encuentran en estado suspendido no están disponibles para el Scheduler. La única forma de entrar en el estado suspendido es mediante una llamada a la función de API vTaskSuspend(). La única manera de salir del estado suspendido es mediante una llamada a las funciones de API vTaskResume() o xTaskResumeFromISR(). La mayoría de las aplicaciones no utilizan el estado suspendido.

Lista (Ready): Se dice que las tareas que no se encuentran en el estado “running”, pero no están bloqueadas ni suspendidas se encuentran en el estado Listo. Están listas para ejecutarse, pero no están actualmente en el estado En ejecución.

c) Las tareas de procesamiento continuo son tareas que no dependen de ningún evento, por lo que siempre pueden entrar en el estado de ejecución, por lo tanto, estas tareas están

limitadas a ser de prioridad baja para que no impidan la ejecución de tareas que tengan menor prioridad. Por el contrario, las tareas manejadas por eventos ejecutan un trabajo solo después de que se produzca el evento que la desencadena, y no puede entrar en el estado de ejecución antes de que se produzca el evento, y por lo tanto no están limitados a tener prioridades bajas

d) Las tareas pueden entrar en el estado Bloqueado para esperar dos tipos de eventos:

1. Eventos temporales (relacionados con el tiempo), donde los eventos son la caducidad de un plazo o que se llega a un tiempo absoluto. Por ejemplo, una tarea puede entrar en estado Bloqueado a esperar que pasen 10 milisegundos.
2. Eventos de sincronización, donde los eventos se originan en otra tarea o interrupción. Por ejemplo, una tarea puede entrar en el estado Bloqueado a esperar a que lleguen datos a una cola. Los eventos de sincronización cubren una amplia gama de tipos de eventos.

e) En FreeRTOS, el procesador siempre ejecuta alguna tarea, aunque no haya tareas de usuario listas. Para asegurarse de que al menos una tarea pueda entrar en el estado En ejecución, el programador crea una tarea de inactividad (IDLE TASK) cuando se llama a `vTaskStartScheduler()`. La tarea de inactividad apenas hace algo más que estar en bucle de modo que siempre puede ejecutarse.

La IDLE TASK tiene la prioridad más baja posible (prioridad cero) para asegurarse de que nunca impida que una aplicación de más prioridad entre en el estado ejecución. Sin embargo, no hay nada que le impida crear tareas con la prioridad de las tareas de inactividad y, por lo tanto, de compartirlas. Las funciones principales de la IDLE TASK son:

- Mantener el procesador ocupado cuando no hay otras tareas listas.
- Liberar recursos de tareas eliminadas. Si una aplicación utiliza la función de API `vTaskDelete()`, es fundamental que la tarea de inactividad no se quede sin tiempo de procesamiento. Esto se debe a que la tarea de inactividad es responsable de limpiar los recursos de kernel después de que se elimine una tarea.
- Medir la cantidad de capacidad de procesamiento libre.
- Poner el procesador en modo de bajo consumo, lo que proporciona un método automático y sencillo de ahorrar energía cuando no es necesario procesar aplicaciones.

3) Una cola puede contener un número finito de elementos de datos de tamaño fijo. El número máximo de elementos que puede contener una cola es su longitud. Tanto la longitud como el tamaño de cada elemento de datos se establecen en el momento de crear la cola. Normalmente, las colas se utilizan como búferes FIFO (primera en entrar, primera en salir), en las que los datos se escriben al final de la cola y se eliminan de la parte delantera de la cola. Hay dos formas de implementar el comportamiento de la cola: cola por copia, donde los datos que se envían a la cola se copian byte por byte en la cola, y cola por referencia, donde la cola contiene punteros a los datos que se envían a la cola, no los datos propiamente dichos.

4) En este ejemplo, se demuestra cómo se crea una cola, cómo varias tareas envían datos a la cola y cómo se reciben datos de la cola. La cola se crea para almacenar elementos de datos de tipo `int32_t`. Las tareas que envían a la cola no especifican un tiempo de bloqueo, pero sí la tarea que recibe de la cola. La prioridad de las tareas que envían a la cola es menor que la prioridad de la tarea que recibe de la cola. Esto significa que la cola no debería contener nunca más de un elemento, ya que, en el momento en el que los datos se envían a la cola, la tarea de recepción se desbloqueará, tendrá preferencia sobre la tarea de envío y eliminará los datos, dejando la cola vacía de nuevo. El código siguiente muestra la implementación de la tarea que escribe en la cola. Se crean dos instancias de esta tarea: una que escribe de forma continua el valor 100 en la cola y otra que escribe de forma continua el valor 200 en la misma cola. El parámetro de la tarea se utiliza para pasar estos valores a cada instancia de la tarea.

```
static void vSenderTask( void *pvParameters )
```

```
{
    int32_t lValueToSend;

    lValueToSend = ( int32_t ) pvParameters;

    for( ;; )
    {
        xQueueSendToBack( xQueue, &lValueToSend, 0 );
    }

    vTaskDelete( NULL );
}
```

```
static void vReceiverTask( void *pvParameters )
```

```
{
    int32_t lReceivedValue;

    for( ;; )
    {
        xQueueReceive( xQueue, &lReceivedValue, portMAX_DELAY);

        vPrintStringAndNumber( "Received = ", lReceivedValue );
    }

    vTaskDelete( NULL );
}
```

```
QueueHandle_t xQueue;
```

```
int main( void )
```

```

{

xQueue = xQueueCreate( 5, sizeof( int32_t ) );

xTaskCreate( vSenderTask, "Sender1", 1000, ( void * ) 100, 1, NULL );

xTaskCreate( vSenderTask, "Sender2", 1000, ( void * ) 200, 1, NULL );

xTaskCreate( vReceiverTask, "Receiver", 1000, NULL, 2, NULL );

vTaskStartScheduler();

for( ; );

}

```

- **Bloqueo por lectura de cola:** Cuando una tarea intenta leer de una cola, puede especificar un tiempo de bloqueo. Es el tiempo durante el que la tarea se mantendrá en el estado Bloqueado a la espera de que haya datos disponibles en la cola en caso de que esté vacía. Una tarea en el estado Bloqueado, a la espera de que haya disponibles datos en una cola, pasa automáticamente al estado Listo cuando otra tarea o interrupción coloca datos en la cola. La tarea también pasa automáticamente del estado Bloqueado al estado Listo si el tiempo de bloqueo especificado vence antes de que los datos estén disponibles. Las colas pueden tener varios lectores, por lo que es posible que una única cola tenga bloqueada más de una tarea a la espera de datos. En ese caso, solo se desbloquea una tarea cuando los datos vuelven a estar disponibles. La tarea que se desbloquea siempre será la tarea de máxima prioridad que está a la espera de datos. Si las tareas bloqueadas tienen la misma prioridad, se desbloqueará la tarea que lleve esperando datos más tiempo.
- **Bloqueo por escritura de cola:** De la misma forma que en las lecturas de colas, una tarea puede especificar un tiempo de bloqueo al escribir en una cola. En este caso, el tiempo de bloqueo es el tiempo máximo durante el que la tarea debe mantenerse en el estado Bloqueado a la espera de que haya espacio disponible en la cola si la cola ya está llena. Las colas pueden tener varios escritores, por lo que es posible que una cola llena tenga bloqueada más de una tarea a la espera de que se complete una operación de envío. En ese caso, solo se desbloquea una tarea cuando vuelve a haber espacio disponible en la cola. La tarea que se desbloquea siempre será la tarea de máxima prioridad que está a la espera de que se libere espacio. Si las tareas bloqueadas tienen la misma prioridad, se desbloqueará la tarea que lleve más tiempo esperando a que se libere espacio.

5) Para enviar datos compuestos se envían a la cola estructuras con el valor de los datos y el origen de los datos contenidos en los campos de la estructura. Esto no genera conflictos con la definición de la macro del recurso porque la cola ya fue creada con el tamaño exacto del dato compuesto. Si el tamaño de los datos que se almacenan en la cola es grande, es mejor utilizar la cola para transferir punteros a los datos en lugar de copiar y sacar los datos de la cola byte por byte. La transferencia de punteros es más eficaz, tanto en lo que se refiere al tiempo de procesamiento como por la cantidad de RAM necesaria para crear la cola.

6) Se crea una cola de estructuras Data_t para comunicar diferentes tareas del sistema. Esta estructura contiene un valor lDataValue y un tipo enumerado eDataID que indica qué representa ese valor. Una tarea de controlador central para realizar la función del sistema principal, la cual recibe mensajes de la cola y reacciona a cambios o comandos. Una tarea del bus CAN recibe y decodifica mensajes del bus, y luego envía estructuras Data_t a la cola, en este caso, un valor de velocidad del motor. Una tarea HMI, o interfaz hombre-máquina, gestiona entradas del operario. Cuando se introduce un comando, también lo envía al controlador usando una estructura Data_t.

7) Los Semáforos Mutex se utilizan para proteger el acceso a un recurso compartido por varias tareas. En el código a continuación se tienen dos tareas que accederían a un recurso compartido y con el uso de un semáforo mutex se garantiza que solo una tarea utilice dicho recurso a la vez y evitando que haya superposición de mensajes ni condiciones de carrera:

```
// Declaración del mutex
```

```
SemaphoreHandle_t xMutex;
```

```
// Tarea 1
```

```
void Tarea1(void *pvParameters)
```

```
{
    for (;;)
    {
        // Espera hasta obtener el mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Sección crítica (recurso compartido)
            printf("Tarea1 accediendo al recurso compartido\n");
            vTaskDelay(pdMS_TO_TICKS(500)); // Simula uso del recurso
            printf("Tarea1 libera el recurso\n");

            // Libera el mutex
            xSemaphoreGive(xMutex);
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```

// Tarea 2

void Tarea2(void *pvParameters)
{
    for (;;)
    {
        // Espera hasta obtener el mutex
        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE)
        {
            // Sección crítica (recurso compartido)
            printf("Tarea2 accediendo al recurso compartido\n");
            vTaskDelay(pdMS_TO_TICKS(500)); // Simula uso del recurso
            printf("Tarea2 libera el recurso\n");

            // Libera el mutex
            xSemaphoreGive(xMutex);
        }

        vTaskDelay(pdMS_TO_TICKS(800));
    }
}

```

```

int main(void)
{
    // Crear el mutex
    xMutex = xSemaphoreCreateMutex();

    if (xMutex != NULL)
    {
        // Crear las tareas
        xTaskCreate(Tarea1, "Tarea1", 128, NULL, 2, NULL);
    }
}

```



```

xTaskCreate(Tarea2, "Tarea2", 128, NULL, 2, NULL);

// Iniciar el planificador
vTaskStartScheduler();

} else
{
    printf("Error al crear el mutex\n");
}

// Nunca debería llegar aquí
for (;;)
}

```

8) Este fenómeno ocurre cuando una tarea de baja prioridad mantiene un recurso compartido que necesita una tarea de alta prioridad, pero la tarea de alta prioridad no puede continuar hasta que el recurso esté libre. Si una tarea de prioridad media se ejecuta constantemente, puede evitar que la tarea de baja prioridad libere el recurso, lo que genera un bloqueo indirecto de la tarea de alta prioridad. En el código a continuación cuando la tarea de alta prioridad queda bloqueada esperando un recurso que tiene la tarea de baja prioridad, FreeRTOS eleva temporalmente la prioridad de la tarea de baja prioridad al nivel de la tarea de alta prioridad, para que libere el recurso más rápidamente y así se evite el bloqueo, esto debido a que FreeRTOS usa herencia de prioridad automáticamente si se usa un mutex.

```

SemaphoreHandle_t xMutex;

// Simula uso de CPU con retraso
void Delay(const char* name, int count)
{
    for (int i = 0; i < count; i++)
    {
        printf("%s ejecutando...\n", name);
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

// Tarea de baja prioridad que toma el mutex

```

```

void TareaBajaPrioridad(void* pvParameters)
{
    while (1)
    {
        printf("Tarea BAJA: intentando tomar el recurso...\n");
        xSemaphoreTake(xMutex, portMAX_DELAY);
        printf("Tarea BAJA: recurso tomado.\n");

        // Simula una operación larga con el recurso
        Delay("Tarea BAJA", 10);

        printf("Tarea BAJA: liberando el recurso.\n");
        xSemaphoreGive(xMutex);
        vTaskDelay(pdMS_TO_TICKS(1000)); // Espera antes de repetir
    }
}

// Tarea de media prioridad que solo consume CPU
void TareaMediaPrioridad(void* pvParameters)
{
    while (1)
    {
        Delay("Tarea MEDIA", 1);
    }
}

// Tarea de alta prioridad que necesita el recurso
void TareaAltaPrioridad(void* pvParameters)
{
    while (1)
    {
        vTaskDelay(pdMS_TO_TICKS(500)); // Arranca después de la baja
        printf("Tarea ALTA: intentando tomar el recurso...\n");
    }
}

```

```

xSemaphoreTake(xMutex, portMAX_DELAY);

printf("Tarea ALTA: recurso tomado.\n");


Delay("Tarea ALTA", 3);


printf("Tarea ALTA: liberando el recurso.\n");
xSemaphoreGive(xMutex);
}
}

int main(void)
{
    // Crear el mutex con herencia de prioridad habilitada
    xMutex = xSemaphoreCreateMutex();

    if (xMutex == NULL)
    {
        printf("No se pudo crear el mutex.\n");
        return -1;
    }

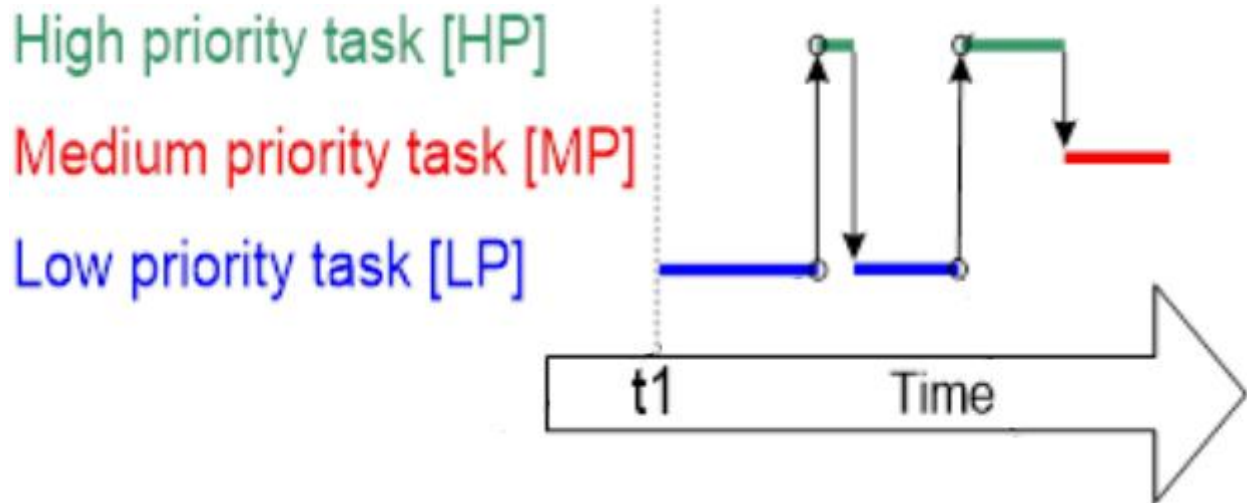

    // Crear tareas con diferentes prioridades
    xTaskCreate(TareaBajaPrioridad, "TareaBaja", 1000, NULL, 1, NULL);
    xTaskCreate(TareaMediaPrioridad, "TareaMedia", 1000, NULL, 2, NULL);
    xTaskCreate(TareaAltaPrioridad, "TareaAlta", 1000, NULL, 3, NULL);


    // Inicia el planificador
    vTaskStartScheduler();


    for (;;)
}

```

9) La tarea de alta prioridad (HP) intenta tomar el mutex pero no puede porque la tarea de baja prioridad (LP) aun no libera el mutex, por lo que HP se bloquea esperando a que el mutex esté disponible. La tarea LP hereda la prioridad de HP para prevenir que la tarea de prioridad media (MP) se ejecute e impida a LP devolver el mutex, y una vez devuelto el mutex LP vuelve a su prioridad original. HP se ejecuta y toma el mutex, y una vez terminada devuelve el mutex y se bloquea dando lugar a que se ejecute MP.



10) Un punto muerto sucede cuando dos tareas quedan bloqueadas simultáneamente esperando el acceso a una cola compartida. En el código siguiente las tareas A y B quedan bloqueadas esperando un recurso (Mutex1, Mutex2) de la otra tarea.

```
SemaphoreHandle_t xMutex1;
```

```
SemaphoreHandle_t xMutex2;
```

```
void TareaA(void *pvParameters) {
```

```
    while (1) {
```

```
        printf("Tarea A: intentando tomar Mutex1...\n");
```

```
        xSemaphoreTake(xMutex1, portMAX_DELAY);
```

```
        printf("Tarea A: tomó Mutex1\n");
```

```
        vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo
```

```
        printf("Tarea A: intentando tomar Mutex2...\n");
```

```
        xSemaphoreTake(xMutex2, portMAX_DELAY); // Aquí queda bloqueada si Mutex2 ya lo tiene
```

```
TareaB
```

```

printf("Tarea A: tomó Mutex2\n");

printf("Tarea A: trabajando con ambos recursos...\n");
vTaskDelay(pdMS_TO_TICKS(100));

xSemaphoreGive(xMutex2);
xSemaphoreGive(xMutex1);
printf("Tarea A: liberó ambos mutex\n");

vTaskDelay(pdMS_TO_TICKS(1000)); // Espera antes de repetir
}
}

void TareaB(void *pvParameters) {
    while (1) {
        printf("Tarea B: intentando tomar Mutex2...\n");
        xSemaphoreTake(xMutex2, portMAX_DELAY);
        printf("Tarea B: tomó Mutex2\n");

        vTaskDelay(pdMS_TO_TICKS(100)); // Simula trabajo

        printf("Tarea B: intentando tomar Mutex1...\n");
        xSemaphoreTake(xMutex1, portMAX_DELAY); // Aquí queda bloqueada si Mutex1 ya lo tiene
TareaA
        printf("Tarea B: tomó Mutex1\n");

        printf("Tarea B: trabajando con ambos recursos...\n");
        vTaskDelay(pdMS_TO_TICKS(100));

        xSemaphoreGive(xMutex1);
        xSemaphoreGive(xMutex2);
    }
}

```

```
printf("Tarea B: liberó ambos mutex\n");

vTaskDelay(pdMS_TO_TICKS(1000)); // Espera antes de repetir
}
}

int main(void) {
    // Crear mutexes

    xMutex1 = xSemaphoreCreateMutex();
    xMutex2 = xSemaphoreCreateMutex();

    if (xMutex1 == NULL || xMutex2 == NULL) {
        printf("Error creando mutexes.\n");
        return -1;
    }

    // Crear tareas

    xTaskCreate(TareaA, "TareaA", 1000, NULL, 2, NULL);
    xTaskCreate(TareaB, "TareaB", 1000, NULL, 2, NULL);

    // Iniciar el planificador

    vTaskStartScheduler();

    for (;;)
}
```